

FIL FRESHER TRAINING · HANDS-ON

Docker, step by step

From “it works on my machine” to a multi-container Python app — one command at a time.

10 steps

Python + Flask app

Docker Engine + Compose v2

YOUR LEARNING PATH

Ten small steps, each one builds on the last

01

Install & verify

02

Pull images

03

Run containers

04

Inspect & debug

05

Write a Dockerfile

06

**Build & run your
app**

07

**Keep data in
volumes**

08

**Connect with
networks**

09

Docker Compose

10

Clean up

WHY DOCKER?

“But it works on my machine...”

Without Docker

- Python 3.9 on your laptop, 3.12 on the server
- pip installs slightly different library versions
- A 20-step setup document nobody follows
- New joiners lose day one to environment setup

With Docker

- App + Python + libraries shipped as one image
- The same image runs on laptop, CI and server
- Setup becomes one command: docker run
- Broke something? Delete it and start fresh in seconds

WHAT IS DOCKER?

Docker packages your app and everything it needs into a **container — a lightweight, isolated process that runs the same everywhere.**

Standard box

Like a shipping container: one shape, whatever is inside.

Runs anywhere

Any machine with Docker can run it — laptop, CI, cloud.

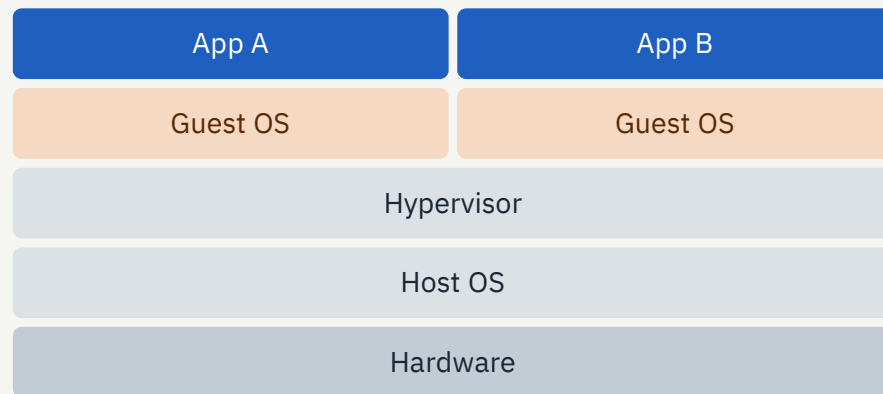
Isolated

Its files, processes and ports don't clash with other apps.

CONTAINERS VS VIRTUAL MACHINES

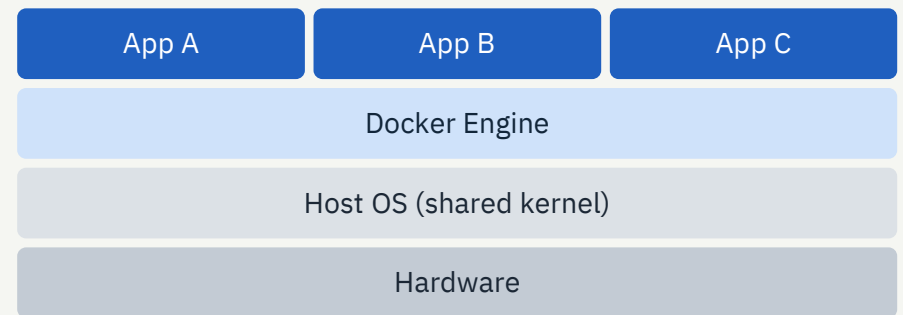
Containers share the host kernel, so they are light

Virtual machine



Each VM boots a full OS: GBs in size, minutes to start.

Container



No guest OS: MBs in size, starts in about a second.

FOUR WORDS TO KNOW

Image, container, Dockerfile, registry



Image

A read-only template: OS files + Python + your code.
Think **class**.



Container

A running instance of an image. Think **object** — many from one image.



Dockerfile

A text recipe of steps that builds an image.



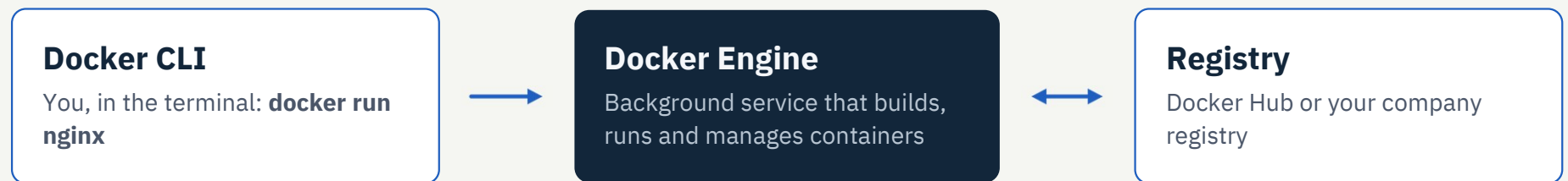
Registry

Where images are stored and shared, e.g. Docker Hub.

Dockerfile → build → Image → run → Container

HOW DOCKER WORKS

You type a command; the engine does the work



What happens on **docker run nginx**

1. The CLI sends the request to the Docker Engine.
2. The engine looks for the nginx image on your machine.
3. Not there? It pulls it from the registry.
4. It creates a container from the image and starts it.

STEP 1 · INSTALL & VERIFY

Get Docker running and say hello

Windows / macOS

Install Docker Desktop, start it, wait for “Engine running”.

Linux

Install Docker Engine and the Compose plugin from docs.docker.com.

`docker version`

Client and Engine versions — both must show

`docker info`

Containers, images and storage on this machine

`docker run hello-world`

Pulls a tiny test image and runs it

You're done when you see “Hello from Docker!” printed in your terminal.

STEP 2 · IMAGES

Pull, list and remove images

```
docker pull python:3.13-slim
```

Download an image from Docker Hub

```
docker images
```

List local images (same as docker image ls)

```
docker rmi <image_id>
```

Remove an image (no container may use it)

```
docker build -t <name:tag> .
```

Build your own image — coming in Step 6

Reading an image name

python = repository, **3.13-slim** = tag (version + variant).

No tag?

Docker assumes **:latest**, which changes over time. Pin tags in real projects.

STEP 3 · RUN CONTAINERS

docker run: four ways you'll use it

```
docker run nginx
```

Start in the foreground (Ctrl+C stops it)

```
docker run -it python:3.13-slim bash
```

Interactive shell inside a container

```
docker run -d -p 8080:80 nginx
```

In the background, host 8080 → container 80

```
docker run -d --name web -p 8080:80 nginx
```

Same, with a name you can reuse

-i keep input open

-t give a terminal

-d detached (background)

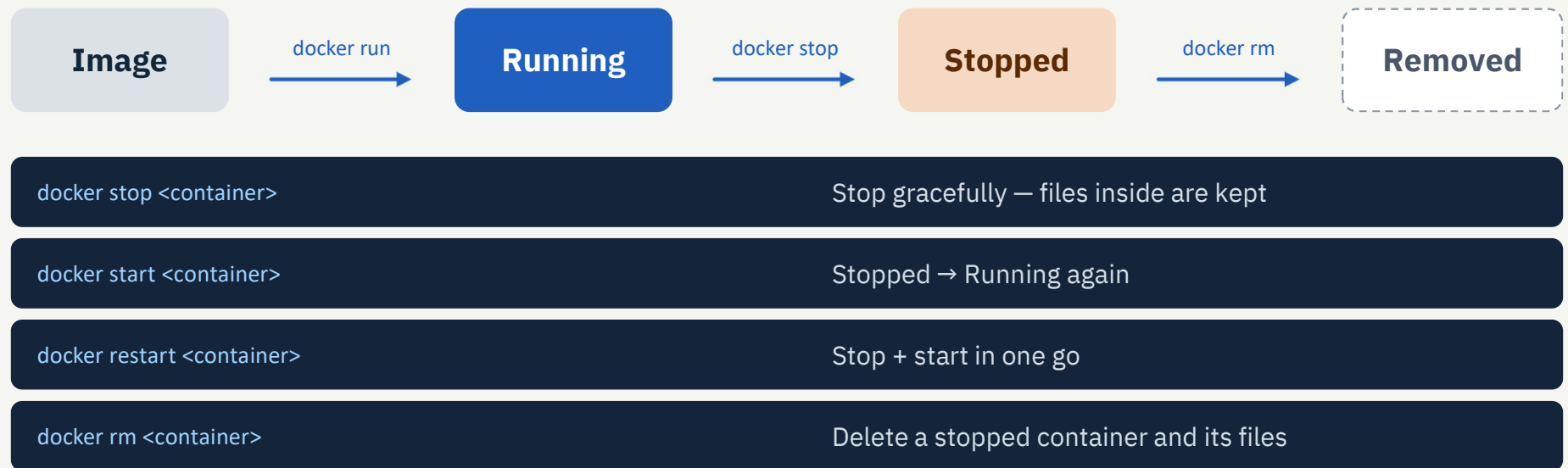
-p host:container port

--name a friendly label

Try it: open <http://localhost:8080> — the nginx welcome page means it worked.

STEP 3 · CONTAINER LIFECYCLE

A container is born, stops, restarts and is removed



STEP 4 · INSPECT & DEBUG

See what's running and look inside it

```
docker ps
```

List running containers

```
docker ps -a
```

All containers, including stopped ones

```
docker logs -f <container>
```

What the app printed; -f keeps following

```
docker exec -it <container> bash
```

Open a shell in a running container

```
docker inspect <container>
```

Full details: ports, mounts, env, IP

Shortcut: use the --name you gave it, or just the first 3–4 characters of the container ID.

STEP 5 · WRITE A DOCKERFILE

Package a small Python API

app.py

```
from flask import Flask
app = Flask(__name__)

@app.get("/")
def hello():
    return {"message": "Hello from Docker!"}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Dockerfile

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

requirements.txt

```
flask>=3.1
```

STEP 5 · DOCKERFILE LINE BY LINE

Each instruction adds one cached layer

Instruction	What it does
FROM python:3.13-slim	Start from the official slim Python base image
WORKDIR /app	Use /app as the working folder inside the image
COPY requirements.txt .	Copy only the dependency list first
RUN pip install ...	Install libraries — a layer that is reused if unchanged
COPY . .	Copy the rest of your source code
EXPOSE 5000	Document the port the app listens on
CMD ["python", "app.py"]	Default command when a container starts

Why copy requirements.txt first? Code changes often, dependencies rarely — so a rebuild after a code edit skips the slow pip install.

STEP 6 · BUILD & RUN YOUR APP

From folder to running API in four commands

```
docker build -t hello-api:1.0 .
```

Build an image from this folder

```
docker run -d -p 8000:5000 --name api hello-api:1.0
```

Start it in the background

```
curl http://localhost:8000
```

Expect {"message": "Hello from Docker!"}

```
docker logs api
```

See Flask's request log

That final dot

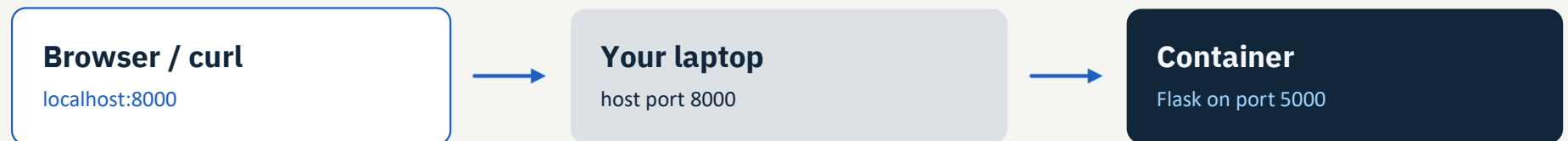
The `.` is the build context: the folder Docker sends to the engine.

Changed the code?

Rebuild with a new tag (1.1), then remove and re-run the container.

STEP 6 · PORT MAPPING

-p HOST:CONTAINER opens a door into the container



-p 8000:5000 → left = your machine, right = inside the container

“Port is already allocated”

Another container uses that host port. Pick a different left side.

“Connection refused”

The app listens on 127.0.0.1. Bind it to 0.0.0.0 inside the container.

STEP 7 · VOLUMES

Keep your data outside the container

Data written inside a container is lost on **docker rm** — so mount storage from outside.

Named volume — for app data

```
docker run -d --name cache \  
-v appdata:/data redis:8
```

Docker manages where it lives. Survives container removal.

Bind mount — for your code

```
docker run -it \  
-v "$(pwd)":/app python:3.13-slim bash
```

Maps your folder in; edits show up live inside.

```
docker volume ls
```

List volumes

```
docker volume rm <volume_name>
```

Delete a volume — and its data

STEP 8 · NETWORKS

Containers talk to each other by name

```
docker network create appnet
```

```
docker run -d --name redis \
  --network appnet redis:8
```

```
docker run -d --name api \
  --network appnet -p 8000:5000 hello-api:1.0
```

network: appnet

api



redis

The api connects to **redis:6379** — Docker's built-in DNS turns the name into an IP.

```
docker network ls
```

List networks

```
docker network inspect appnet
```

See which containers are attached

STEP 9 · DOCKER COMPOSE

Describe the whole app in one file

compose.yaml

```
services:
  api:
    build: .
    ports:
      - "8000:5000"
    depends_on:
      - redis
  redis:
    image: redis:8
    volumes:
      - redisdata:/data
volumes:
  redisdata:
```

services = containers

api is built from our Dockerfile; redis uses a ready image.

Same ideas as Steps 6–8

ports, volumes and names — written once instead of typed every time.

Network for free

Compose creates one network; api reaches redis by its service name.

STEP 9 · COMPOSE COMMANDS

One command starts the whole stack

Write `docker compose` (space), not `docker-compose`. The hyphen version is the retired Compose v1; v2 is built into Docker with the same sub-commands.

```
docker compose up -d --build
```

Build if needed, start all in background

```
docker compose ps
```

Status of each service

```
docker compose logs -f api
```

Follow one service's logs (omit api for all)

```
docker compose exec api bash
```

Shell inside a running service

```
docker compose restart
```

Restart services (no rebuild)

```
docker compose down
```

Stop and remove containers + network

STEP 9 · SCALING

Run several copies of a service

```
docker compose up -d --scale api=3
```

Three containers of the api service

The catch

With a fixed **"8000:5000"**, only one copy can own host port 8000. The others fail: “port is already allocated”.

Fix for practice

Give a port range so each copy gets its own:

```
ports:  
  - "8000-8002:5000"
```

In real systems a load balancer (e.g. nginx) sits in front and spreads traffic across the copies.

STEP 10 · CLEAN UP

Images and stopped containers eat disk — tidy up

`docker system df`

How much space Docker is using

`docker container prune`

Remove all stopped containers

`docker image prune`

Remove dangling (untagged) images

`docker system prune`

Stopped containers, unused networks, dangling images, build cache

Prune is permanent. Adding `-a` also removes every unused image; `--volumes` deletes volume data. Read the prompt before you type `y`.

GOOD HABITS

Six habits to build from day one

Pin versions

python:3.13-slim, not python:latest.

Start slim

Smaller images build faster and carry fewer vulnerabilities.

Add .dockerignore

Keep .git, .venv and __pycache__ out of the image.

One process each

api and redis are separate services, not one container.

No secrets inside

Pass passwords and keys at runtime, never in the Dockerfile.

Rebuild, don't patch

Change the Dockerfile and rebuild — never hand-edit a running container.

HANDS-ON LAB · 45 MINUTES

Ship hello-api end to end

1. Run **docker run hello-world** and read what it tells you.
2. Run nginx on port 8080, open it in a browser, then stop and remove it.
3. Create app.py, requirements.txt and the Dockerfile from Step 5.
4. Build hello-api:1.0, run it on port 8000 and call it with curl.
5. Change the message, rebuild as 1.1 and spot the CACHED layers.
6. Write compose.yaml with api + redis and run **docker compose up -d**.
7. Use **docker compose logs** and **exec** to look inside.
8. Tear down with **docker compose down**, then check **docker system df**.

Stretch goal: make the API count visits in redis and show that the count survives docker compose down and up.

CHEATSHEET RECAP

The commands you'll use every week

Images

```
docker pull python:3.13-slim
docker images
docker build -t name:tag .
docker rmi <image>
```

Containers

```
docker run -d -p 8080:80 --name web nginx
docker ps -a
docker logs -f web · docker exec -it web bash
docker stop / start / rm web
```

Data, network, cleanup

```
docker volume ls / rm <volume>
docker network ls / inspect <net>
docker system df
docker system prune
```

Compose

```
docker compose up -d --build
docker compose ps · logs -f
docker compose exec api bash
docker compose down
```

WRAP-UP

Questions?

You can now run, build, connect and compose containers. Next: using these images in CI pipelines.

Dockerfile → build → image → run → container